

UNIVERSIDAD NACIONAL DE LA RIOJA
Cátedra: Sistemas Operativos Avanzados
ISI / LSI



Trabajo Práctico N° 1: Gestión de Procesos en GNU/Linux

Modalidad de entrega: El trabajo práctico deberá ser realizado en forma individual y entregado en un único archivo en formato PDF. El documento debe contener las respuestas a cada pregunta de forma clara y ordenada. Para las consignas que requieran el uso de comandos, se debe incluir tanto el comando ejecutado como la salida de texto relevante (copiar y pegar desde la terminal en un bloque de código) y/o las capturas de pantalla solicitadas.

Fecha de Entrega: 08-10-2025

Objetivos:

- **Comprender** y diferenciar los conceptos fundamentales de programa, proceso y su ciclo de vida, incluyendo estados especiales como el estado zombie.
- **Identificar** el rol del gestor de sistemas systemd como proceso raíz y su importancia en la arquitectura de un sistema GNU/Linux moderno.
- **Adquirir** la habilidad práctica para utilizar comandos (ps, pstree, htop) para observar, listar y analizar los procesos en ejecución, su estado y su jerarquía.
- **Demostrar** la capacidad de interactuar directamente con el sistema de archivos virtual /proc para extraer información específica del sistema.
- **Aplicar** comandos (nice) para influir en la planificación de procesos mediante la gestión de sus prioridades.
- **Evaluar** y comparar herramientas de monitoreo del sistema, reconociendo las ventajas y diferencias funcionales entre ellas.

UNIVERSIDAD NACIONAL DE LA RIOJA

Cátedra: Sistemas Operativos Avanzados

ISI / LSI



Consignas:

1. Fundamentos Teóricos: Proceso vs. Programa

Explique con sus propias palabras cuál es la diferencia fundamental entre un programa y un proceso. Describa brevemente las principales secciones que componen el espacio de memoria de un proceso.

2. Ciclo de Vida: El Proceso Zombie

Defina qué es un proceso zombie. Explique por qué es una parte necesaria del ciclo de vida de un proceso y describa la única condición bajo la cual un proceso podría permanecer en estado zombie de forma indefinida.

3. Arquitectura del Sistema: El Rol de systemd

Describa la función de systemd como proceso con PID 1 en un sistema Debian moderno. Mencione y explique dos ventajas significativas que ofrece systemd en comparación con el antiguo sistema de inicio init.

4. Observación de Procesos con ps y grep

Ejecute el comando necesario para listar todos los procesos del sistema en formato detallado. Redirija la salida a un archivo llamado **procesos.txt**. Luego, utilizando los comandos **grep** y **wc**, determine cuántos procesos están siendo ejecutados por el usuario root. Adjunte los comandos exactos que utilizó para lograrlo.

5. Jerarquía de Procesos con pstree

Genere una vista de la jerarquía de procesos del sistema. Identifique el Proceso Padre (PPID) de su shell (bash o similar) actual y trace su linaje hacia atrás hasta llegar a systemd. Puede describir la rama del árbol o adjuntar una captura de pantalla resaltando la jerarquía.

6. Monitoreo en Tiempo Real con htop

Ejecute el comando htop y ordene la vista por uso de memoria (MEM%). Adjunte una captura de pantalla de la vista completa de htop y explique brevemente qué representan las columnas PID, NI (nice), y STAT (estado).

7. Identificación de Procesos Daemons

Utilice el comando ps con las opciones adecuadas para listar todos los procesos que no están asociados a una terminal. De esa lista, filtre la salida para encontrar el PID y el comando completo del demonio de cron o de sshd. Adjunte el comando completo que utilizó.

UNIVERSIDAD NACIONAL DE LA RIOJA
Cátedra: Sistemas Operativos Avanzados
ISI / LSI



8. Gestión de Prioridades con nice

Ejecute un comando (ej: `sleep 300`) en segundo plano, asignándole desde el inicio un valor `nice` de 12. Verifique la prioridad del proceso utilizando `ps` o `htop` y adjunte una captura de pantalla donde se vea claramente el proceso y su valor `NI`.

9. Exploración del Sistema de Archivos /proc

Utilizando únicamente los comandos `cat` y `grep` sobre el sistema de archivos `/proc` (sin usar `lscpu`, `free`, etc.), obtenga la siguiente información:

- a) El modelo exacto de la CPU (`model name`).
- b) La cantidad total de memoria RAM del sistema (`MemTotal`).

Adjunte los comandos completos que utilizó.

10. Comparativa de Herramientas: top vs. htop

Investigue y describa tres diferencias funcionales o de interfaz entre los comandos `top` y `htop`. Para cada diferencia, explique brevemente por qué la implementación en `htop` puede ser considerada una mejora para el usuario.
